

Seguro Viagem – Gestão de Apólices

Documentação técnica do código

Carlos Guilherme Fontes Pereira

Teste técnico – CORIS

24 de setembro de 2026

Resumo

Este documento descreve o funcionamento de uma aplicação web para gestão de apólices de seguro viagem, construída com Laravel 12 (PHP 8.3), React 18 e MySQL 8, com implantação prevista na Microsoft Azure. São apresentados o modelo de dados, as regras de negócio, a organização do código em camadas, a API e os testes automatizados, com o objetivo de permitir que o leitor entenda o sistema e consiga modificá-lo.

Sumário

1	Introdução	3
1.1	Tecnologias	3
2	Modelo de dados	3
3	Regras de negócio	4
3.1	Cálculo do prêmio	4
3.2	Validações e demais regras	4
4	Arquitetura do backend	4
4.1	Exemplo: o que acontece ao emitir uma apólice	5
4.2	Rotas	5
4.3	Validação	5
4.4	Controller	7
4.5	Service	8
4.6	Calculadora do prêmio	11
4.7	Models e Enums	12
5	Dashboard	12
5.1	Tabelas	13
5.2	Fórmulas	13
5.3	Organização	13

5.4	Dados de demonstração	13
6	SOLID e Clean Code	14
7	API	14
8	Frontend	15
9	Testes	15
10	Execução e implantação	15
10.1	Com Docker	15
10.2	Na Azure	16
11	Trabalhos futuros	16

1 Introdução

A aplicação permite a um operador de seguradora **cadastrar, consultar, editar e excluir** apólices de seguro viagem. Ao preencher o formulário, o operador informa os dados do segurado, o destino, o plano e o período da viagem; o sistema então calcula o *prêmio*, isto é, o valor a ser pago pelo seguro.

O projeto está dividido em duas partes independentes que conversam por HTTP/JSON:

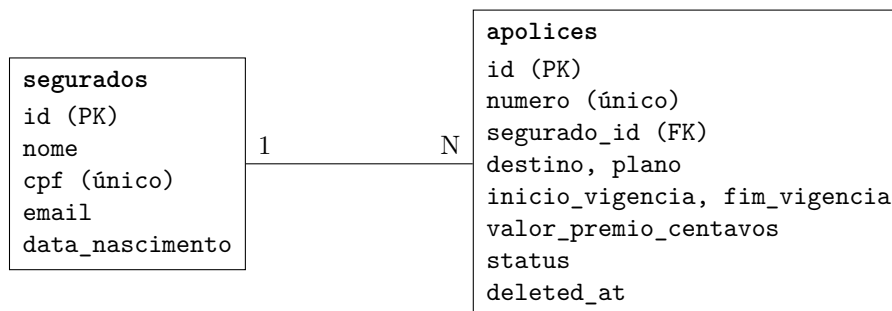
- **backend** (`backend/`): API REST em Laravel, responsável pelas regras e pelo banco;
- **frontend** (`frontend/`): interface em React, que apenas exibe dados e envia formulários.

1.1 Tecnologias

Camada	Tecnologia
Backend	PHP 8.3, Laravel 12
Frontend	React 18, Vite, React Router
Banco de dados	MySQL 8 (SQLite no desenvolvimento local)
Testes	PHPUnit
Ambiente	Docker Compose
Nuvem	Microsoft Azure

2 Modelo de dados

O banco possui duas tabelas com relacionamento **1:N**: um segurado pode ter várias apólices, e cada apólice pertence a um único segurado.



Três decisões merecem destaque:

1. **Valores em centavos.** O prêmio é guardado como número inteiro de centavos (32370 = R\$ 323,70). Números de ponto flutuante não representam decimais com exatidão (em float, $0,1 + 0,2 = 0,30000000000000004$), o que é inaceitável para valores financeiros.
2. **Segurado separado da apólice.** O CPF é único, então uma mesma pessoa que contrata outro seguro é reaproveitada, sem duplicar dados.
3. **Exclusão lógica.** Excluir uma apólice apenas preenche a coluna `deleted_at`. O registro some das consultas, mas continua no banco, pois contratos de seguro precisam ser mantidos para auditoria.

As tabelas são criadas pelas *migrations* em `backend/database/migrations/`.

3 Regras de negócio

3.1 Cálculo do prêmio

$\text{prêmio} = \text{diária do plano} \times \text{dias} \times \text{fator do destino} \times \text{fator da idade}$

		Destino	Fator		
Plano	Diária			Idade	Fator
Essencial	R\$ 12,90	Nacional	50%	até 59	100%
Plus	R\$ 24,90	América do Sul	100%	60 a 74	160%
Premium	R\$ 39,90	Europa	130%	75 ou mais	250%
		América do Norte	140%		
		Ásia, África, Oceania	150%		

Os dias incluem o primeiro e o último (de 01/10 a 10/10 são 10 dias), e a idade é calculada na data de início da viagem.

Exemplo. Plano Plus, Europa, 10 dias, segurado de 31 anos:

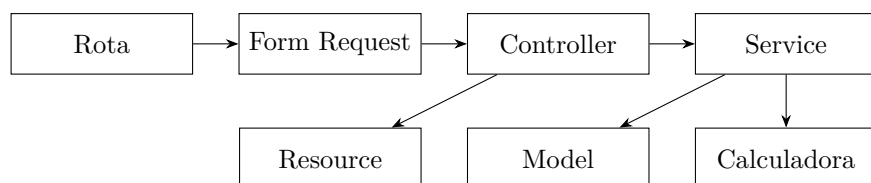
$$2490 \times 10 = 24900 \rightarrow 24900 \times 1,30 = 32370 \rightarrow 32370 \times 1,00 = 32370 \text{ centavos} = \text{R\$ } 323,70.$$

3.2 Validações e demais regras

- O CPF deve ter dígitos verificadores válidos.
- O fim da vigência não pode ser anterior ao início, e a viagem tem no máximo 365 dias.
- O início da vigência não pode ser anterior a hoje. Na edição, a regra só é aplicada se a data de início mudou, para que uma apólice já em vigor continue editável.
- Toda apólice recebe um número único no formato CRS-2026-XXXXXXX.
- Uma apólice cancelada só pode ser alterada para voltar a ficar ativa.

4 Arquitetura do backend

O backend é organizado em camadas, e cada classe tem uma única responsabilidade:



Camada	Responsabilidade
Rota (<code>routes/api.php</code>)	associa a URL ao método do controller
Form Request (<code>app/Http/Requests</code>)	valida a entrada; se falhar, responde 422
Controller (<code>app/Http/Controllers/Api</code>)	recebe a requisição e devolve a resposta HTTP
Service (<code>app/Services/ApoliceService.php</code>)	aplica as regras de negócio
Calculadora (<code>app/Services/Premio</code>)	calcula o prêmio
Model (<code>app/Models</code>)	representa as tabelas (Eloquent)
Resource (<code>app/Http/Resources</code>)	define o formato do JSON de saída
Enums (<code>app/Enums</code>)	planos, destinos e status, com seus valores

4.1 Exemplo: o que acontece ao emitir uma apólice

1. O React envia POST /api/apolices com os dados do formulário em JSON.
2. O `SalvarApoliceRequest` valida os campos. Havendo erro, a API responde 422 com uma mensagem por campo, e o fluxo termina aqui.
3. O `ApoliceController` chama `ApoliceService::criar`.
4. O service, dentro de uma transação, cria ou atualiza o segurado pelo CPF, calcula o prêmio, gera o número da apólice e a grava.
5. O `ApoliceResource` converte a apólice em JSON, e a resposta é 201 Created.

4.2 Rotas

Código 1: routes/api.php

```
1 <?php
2
3 use App\Http\Controllers\Api\ApoliceController;
4 use App\Http\Controllers\Api\DashboardController;
5 use App\Http\Controllers\Api\OpcoesController;
6 use Illuminate\Support\Facades\Route;
7
8 Route::get('/health', fn () => ['status' => 'ok']);
9 Route::get('/opcoes', OpcoesController::class);
10
11 Route::get('/apolices/resumo', [ApoliceController::class, 'resumo']);
12 Route::post('/apolices/cotacao', [ApoliceController::class, 'cotacao']);
13 Route::apiResource('apolices', ApoliceController::class)
14     ->parameters(['apolices' => 'apolice'])
15     ->whereNumber('apolice');
16
17 Route::get('/dashboard/{visao}', DashboardController::class)
18     ->whereIn('visao', ['visao-geral', 'marketing', 'comercial', 'sinistros']);
```

As rotas `resumo` e `cotacao` vêm antes do `apiResource`, e `whereNumber` impede que `/apolices/resumo` seja interpretado como `/apolices/{id}`.

4.3 Validação

`CotacaoRequest` contém as regras comuns à cotação e ao cadastro. `SalvarApoliceRequest` herda essas regras e acrescenta o campo `status` e a verificação da data de início. Validações que envolvem mais de um campo ficam no método `after()`.

Código 2: app/Http/Requests/CotacaoRequest.php

```
1 <?php
2
3 namespace App\Http\Requests;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use App\Rules\Cpf;
8 use Illuminate\Foundation\Http\FormRequest;
9 use Illuminate\Support\Carbon;
```

```

10 use Illuminate\Validation\Rule;
11 use Illuminate\Validation\Validator;
12
13 class CotacaoRequest extends FormRequest
14 {
15     public const VIGENCIA_MAXIMA_DIAS = 365;
16
17     public function rules(): array
18     {
19         return [
20             'seguradoNome' => ['required', 'string', 'min:3', 'max:120'],
21             'seguradoCpf' => ['required', new Cpf],
22             'seguradoEmail' => ['required', 'email', 'max:150'],
23             'seguradoNascimento' => ['required', 'date_format:Y-m-d', '
before_or_equal:today'],
24             'destino' => ['required', Rule::in(array_column(Destino::cases(), '
value'))],
25             'plano' => ['required', Rule::in(array_column(Plano::cases(), 'value'))
],
26             'inicioVigencia' => ['required', 'date_format:Y-m-d'],
27             'fimVigencia' => ['required', 'date_format:Y-m-d', 'after_or_equal:
inicioVigencia'],
28         ];
29     }
30
31     public function after(): array
32     {
33         return [
34             function (Validator $validator) {
35                 if ($validator->errors()->hasAny(['inicioVigencia', 'fimVigencia'])
) {
36                     return;
37                 }
38
39                 $dias = Carbon::parse($this->input('inicioVigencia'))->diffInDays(
$this->input('fimVigencia')) + 1;
40
41                 if ($dias > self::VIGENCIA_MAXIMA_DIAS) {
42                     $validator->errors()->add('fimVigencia', 'A vigência máxima é
de '.self::VIGENCIA_MAXIMA_DIAS.' dias.');
```

```

        posterior ao início.',
60         'destino.in' => 'Destino inválido.',
61         'plano.in' => 'Plano inválido.',
62         'status.in' => 'Status inválido.',
63     ];
64 }
65 }

```

O CPF é validado pela regra `app/Rules/Cpf.php`: cada dígito verificador é obtido multiplicando os dígitos anteriores por pesos decrescentes, somando e calculando $(10 \cdot \text{soma}) \bmod 11$ (o resultado 10 vale 0).

4.4 Controller

O controller não contém regra de negócio: apenas repassa os dados validados ao service e escolhe o código HTTP da resposta.

Código 3: `app/Http/Controllers/Api/ApoliceController.php`

```

1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use App\Http\Requests\CotacaoRequest;
7 use App\Http\Requests\SalvarApoliceRequest;
8 use App\Http\Resources\ApoliceResource;
9 use App\Models\Apolice;
10 use App\Services\ApoliceService;
11 use Illuminate\Http\JsonResponse;
12 use Illuminate\Http\Request;
13 use Illuminate\Http\Resources\Json\AnonymousResourceCollection;
14 use Illuminate\Http\Response;
15
16 class ApoliceController extends Controller
17 {
18     public function __construct(private readonly ApoliceService $service) {}
19
20     public function index(Request $request): AnonymousResourceCollection
21     {
22         return ApoliceResource::collection(
23             $this->service->listar($request->query('busca'), $request->query('
24             status'))
25         );
26     }
27
28     public function resumo(): JsonResponse
29     {
30         return response()->json($this->service->resumo());
31     }
32
33     public function show(Apolice $apolice): ApoliceResource
34     {
35         return new ApoliceResource($apolice->load('segurado'));
36     }
37
38     public function store(SalvarApoliceRequest $request): JsonResponse

```

```

38     {
39         $apolice = $this->service->criar($request->validated());
40
41         return (new ApoliceResource($apolice))->response()->setStatusCode(201);
42     }
43
44     public function update(SalvarApoliceRequest $request, Apolice $apolice):
45     ApoliceResource
46     {
47         return new ApoliceResource($this->service->atualizar($apolice, $request->
48         validated()));
49     }
50
51     public function destroy(Apolice $apolice): Response
52     {
53         $this->service->excluir($apolice);
54
55         return response()->noContent();
56     }
57
58     public function cotacao(CotacaoRequest $request): JsonResponse
59     {
60         return response()->json($this->service->cotar($request->validated()));
61     }
62 }

```

4.5 Service

É a classe central do sistema. Observe que o construtor recebe a *interface* `CalculadoraPremio`, e não uma classe concreta.

Código 4: `app/Services/ApoliceService.php`

```

1 <?php
2
3 namespace App\Services;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use App\Enums>StatusApolice;
8 use App\Models\Apolice;
9 use App\Models\Segurado;
10 use App\Services\Premio\CalculadoraPremio;
11 use Illuminate\Contracts\Pagination\LengthAwarePaginator;
12 use Illuminate\Support\Carbon;
13 use Illuminate\Support\Facades\DB;
14 use Illuminate\Support\Str;
15 use Illuminate\Validation\ValidationException;
16
17 class ApoliceService
18 {
19     public const POR_PAGINA = 10;
20
21     public function __construct(private readonly CalculadoraPremio $calculadora) {}
22
23     public function listar(?string $busca, ?string $status): LengthAwarePaginator
24     {

```

```

25         return Apolice::with('segurado')
26             ->buscar($busca)
27             ->when(StatusApolice::tryFrom((string) $status), fn ($query, $status)
=> $query->where('status', $status))
28             ->latest()
29             ->latest('id')
30             ->paginate(self::POR_PAGINA);
31     }
32
33     public function resumo(): array
34     {
35         $ativas = Apolice::where('status', StatusApolice::Ativa);
36
37         return [
38             'total' => Apolice::count(),
39             'ativas' => $ativas->count(),
40             'premioAtivasCentavos' => (int) $ativas->sum('valor_premio_centavos'),
41         ];
42     }
43
44     public function criar(array $dados): Apolice
45     {
46         return DB::transaction(function () use ($dados) {
47             $apolice = Apolice::create([
48                 ...$this->dadosDaApolice($dados),
49                 'numero' => $this->gerarNumero(),
50                 'segurado_id' => $this->salvarSegurado($dados)->id,
51                 'status' => StatusApolice::Ativa,
52             ]);
53
54             return $apolice->load('segurado');
55         });
56     }
57
58     public function atualizar(Apolice $apolice, array $dados): Apolice
59     {
60         $novoStatus = StatusApolice::tryFrom($dados['status'] ?? '') ?? $apolice->
status;
61
62         if ($apolice->status === StatusApolice::Cancelada && $novoStatus ===
StatusApolice::Cancelada) {
63             throw ValidationException::withMessages([
64                 'status' => 'Apólice cancelada não pode ser alterada. Reative-a
primeiro.',
65             ]);
66         }
67
68         return DB::transaction(function () use ($apolice, $dados, $novoStatus) {
69             $apolice->update([
70                 ...$this->dadosDaApolice($dados),
71                 'segurado_id' => $this->salvarSegurado($dados)->id,
72                 'status' => $novoStatus,
73             ]);
74
75             return $apolice->load('segurado');
76         });
77     }
78

```

```

79 public function excluir(Apolice $apolicy): void
80 {
81     $apolicy->delete();
82 }
83
84 public function cotar(array $dados): array
85 {
86     $inicio = Carbon::parse($dados['inicioVigencia']);
87     $fim = Carbon::parse($dados['fimVigencia']);
88
89     return [
90         'valorPremioCentavos' => $this->premio($dados),
91         'dias' => (int) $inicio->diffInDays($fim) + 1,
92     ];
93 }
94
95 private function dadosDaApolice(array $dados): array
96 {
97     return [
98         'destino' => $dados['destino'],
99         'plano' => $dados['plano'],
100         'inicio_vigencia' => $dados['inicioVigencia'],
101         'fim_vigencia' => $dados['fimVigencia'],
102         'valor_premio_centavos' => $this->premio($dados),
103     ];
104 }
105
106 private function salvarSegurado(array $dados): Segurado
107 {
108     return Segurado::updateOrCreate(
109         ['cpf' => preg_replace('/\D/', '', $dados['seguradoCpf'])],
110         [
111             'nome' => trim($dados['seguradoNome']),
112             'email' => mb_strtolower(trim($dados['seguradoEmail'])),
113             'data_nascimento' => $dados['seguradoNascimento'],
114         ],
115     );
116 }
117
118 private function premio(array $dados): int
119 {
120     return $this->calculadora->calcular(
121         Plano::from($dados['plano']),
122         Destino::from($dados['destino']),
123         Carbon::parse($dados['inicioVigencia']),
124         Carbon::parse($dados['fimVigencia']),
125         Carbon::parse($dados['seguradoNascimento']),
126     );
127 }
128
129 private function gerarNumero(): string
130 {
131     do {
132         $numero = 'CRS-'.now()->year.'-'.Str::upper(bin2hex(random_bytes(4)));
133     } while (Apolice::withTrashed()->where('numero', $numero)->exists());
134
135     return $numero;
136 }

```

Pontos importantes:

- `DB::transaction` garante que seguro e apólice sejam gravados juntos, ou nenhum dos dois.
- `updateOrCreate` reaproveita o seguro quando o CPF já existe.
- `gerarNumero` consulta também as apólices excluídas (`withTrashed`), para nunca repetir um número.

4.6 Calculadora do prêmio

Código 5: `app/Services/Premio/CalculadoraPremio.php`

```

1 <?php
2
3 namespace App\Services\Premio;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use Carbon\CarbonInterface;
8
9 interface CalculadoraPremio
10 {
11     /** Retorna o valor do prêmio em centavos. */
12     public function calcular(
13         Plano $plano,
14         Destino $destino,
15         CarbonInterface $inicio,
16         CarbonInterface $fim,
17         CarbonInterface $dataNascimento,
18     ): int;
19 }
```

Código 6: `app/Services/Premio/CalculadoraPremioViagem.php`

```

1 <?php
2
3 namespace App\Services\Premio;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use Carbon\CarbonInterface;
8
9 class CalculadoraPremioViagem implements CalculadoraPremio
10 {
11     public function calcular(
12         Plano $plano,
13         Destino $destino,
14         CarbonInterface $inicio,
15         CarbonInterface $fim,
16         CarbonInterface $dataNascimento,
17     ): int {
18         $dias = (int) $inicio->diffInDays($fim) + 1;
19         $idade = (int) $dataNascimento->diffInYears($inicio);
```

```

20
21     $valor = $plano->diariaCentavos() * $dias;
22     $valor = $this->aplicarPercentual($valor, $destino->percentualRisco());
23
24     return $this->aplicarPercentual($valor, $this->percentualIdade($idade));
25 }
26
27 private function percentualIdade(int $idade): int
28 {
29     return match (true) {
30         $idade >= 75 => 250,
31         $idade >= 60 => 160,
32         default => 100,
33     };
34 }
35
36 private function aplicarPercentual(int $centavos, int $percentual): int
37 {
38     return intdiv($centavos * $percentual + 50, 100);
39 }
40 }

```

O arredondamento usa apenas inteiros: `intdiv(centavos * percentual + 50, 100)`. Somar 50 antes da divisão inteira arredonda para o centavo mais próximo.

A ligação entre a interface e a implementação é feita em `app/Providers/AppServiceProvider.php`:

```

$this->app->bind(CalculadoraPremio::class, CalculadoraPremioViagem::class);

```

4.7 Models e Enums

Segurado possui `hasMany(Apolice)` e `Apolice` possui `belongsTo(Segurado)`. A apólice usa `SoftDeletes` (exclusão lógica) e `casts`, que convertem automaticamente as colunas em enums e datas. O `scope` `buscar` procura um termo no número da apólice e no nome, e-mail ou CPF do segurado.

Os valores de negócio ficam nos enums. Por exemplo, a diária de cada plano:

```

public function diariaCentavos(): int
{
    return match ($this) {
        self::Essencial => 1290,
        self::Plus => 2490,
        self::Premium => 3990,
    };
}

```

Para criar um novo plano basta adicionar um `case`: o frontend obtém a lista pela rota `/api/opcoes` e passa a exibi-lo sem alteração.

5 Dashboard

A tela *Dashboard* reúne indicadores de vendas, marketing, sinistros e atendimento, com filtro de período (30 dias, 90 dias, 12 e 24 meses) e quatro visões: visão geral, marketing, comercial e sinistros/atendimento. O período e a visão ficam na URL, o que permite compartilhar o link.

5.1 Tabelas

As tabelas antigas não foram alteradas diretamente: foram criadas *migrations* novas.

Tabela	Conteúdo
<code>canais</code>	site, agências, corretores, parceiros e app
<code>campanhas</code>	campanhas de marketing, com UTM, orçamento e investimento
<code>cotacoes</code>	toda simulação de preço, convertida em apólice ou abandonada
<code>funil_eventos</code>	um registro para cada etapa que o cliente passou na cotação
<code>sinistros</code>	ocorrências cobertas pela apólice: valor reclamado, valor pago e status
<code>atendimentos</code>	contatos com a central 24h: canal, tempo de espera, SLA e nota NPS

A apólice ganhou `canal_id` e `campanha_id`, opcionais, para saber de onde veio cada venda.

5.2 Fórmulas

Todas as contas ficam na classe `app/Services/Dashboard/Metricas.php`, para que todas as telas usem a mesma fórmula:

Métrica	Fórmula
Ticket médio	$\text{prêmio emitido} \div \text{apólices}$
Conversão	$\text{cotações convertidas} \div \text{cotações}$
Prêmio ganho	$\text{prêmio} \times (\text{dias de viagem dentro do período} \div \text{dias da viagem})$
Sinistralidade	$\text{custo dos sinistros} \div \text{prêmio ganho}$
Frequência	$\text{sinistros avisados} \div \text{apólices emitidas}$
Taxa de negativa	$\text{sinistros negados} \div \text{sinistros finalizados}$
ROI da campanha	$(\text{prêmio gerado} - \text{investimento}) \div \text{investimento}$
NPS	$\% \text{ de promotores (notas 9 e 10)} - \% \text{ de detratores (notas 0 a 6)}$

A sinistralidade usa o **prêmio ganho**, e não o emitido: uma apólice vendida hoje para uma viagem daqui a dois meses ainda não correu risco. Usar o prêmio emitido distorce o indicador em períodos curtos.

5.3 Organização

O `DashboardController` valida o período e chama o `DashboardService`, que tem um método por visão e monta os números com consultas agrupadas (`GROUP BY`). A classe `Periodo` calcula o intervalo escolhido e o período anterior, de mesmo tamanho, usado na comparação dos indicadores. No frontend, cada visão é um componente em `src/pages/dashboard/`, e os gráficos usam a biblioteca `Recharts`.

5.4 Dados de demonstração

O `DashboardSeeder` gera 24 meses de histórico, com cerca de 2.800 apólices, 12.800 cotações, 180 sinistros e 1.800 atendimentos, com sazonalidade (julho, dezembro e janeiro mais fortes), crescimento de 12% ao ano e conversão menor no celular. Ele usa uma semente fixa (`mt_srand(2026)`), então gera sempre os mesmos dados.

6 SOLID e Clean Code

Princípio	Aplicação
S – Responsabilidade única	validação, HTTP, regra de negócio, cálculo e formatação em classes distintas
O – Aberto/fechado	nova regra de preço é uma nova classe que implementa <code>CalculadoraPremio</code> ; novo plano é um novo <code>case</code>
L – Substituição de Liskov	qualquer implementação da calculadora pode substituir a atual
I – Segregação de interfaces	a interface da calculadora possui um único método
D – Inversão de dependência	o service depende da interface; o provider escolhe a implementação

Quanto a Clean Code: os nomes seguem o vocabulário do negócio (apólice, segurado, prêmio, vigência), os métodos são curtos e os valores fixos ficam em enums ou constantes (`VIGENCIA_MAXIMA_DIAS`, `POR_PAGINA`).

7 API

Método	Rota	Descrição
GET	<code>/api/apolices</code>	lista paginada (<code>busca</code> , <code>status</code> , <code>page</code>)
GET	<code>/api/apolices/resumo</code>	totais da tela inicial
GET	<code>/api/apolices/{id}</code>	detalhe
POST	<code>/api/apolices</code>	cadastro (201)
PUT	<code>/api/apolices/{id}</code>	edição, com recálculo do prêmio
DELETE	<code>/api/apolices/{id}</code>	exclusão lógica (204)
POST	<code>/api/apolices/cotacao</code>	calcula o prêmio sem salvar
GET	<code>/api/opcoes</code>	planos, destinos e status
GET	<code>/api/dashboard/{visao}</code>	números de uma visão da dashboard (<code>periodo</code>)

Exemplo de requisição e resposta de erro:

```
POST /api/apolices
{ "seguradoNome": "Carlos Pereira", "seguradoCpf": "123.456.789-00",
  "seguradoEmail": "carlos@email.com", "seguradoNascimento": "1995-05-10",
  "destino": "europa", "plano": "ouro",
  "inicioVigencia": "2026-10-01", "fimVigencia": "2026-10-10" }

422 Unprocessable Content
{ "message": "CPF inválido. (e mais 1 erro)",
  "errors": { "seguradoCpf": ["CPF inválido."], "plano": ["Plano inválido."] } }
```

O banco usa nomes em `snake_case` e a API responde em `camelCase`; essa conversão é feita pelo `ApoliceResource`.

8 Frontend

Pasta	Conteúdo
src/pages	telas: lista, detalhe, cadastro e edição
src/components	formulário, paginação, diálogo de confirmação, notificações
src/api	client.js (função request) e apolices.js (uma função por endpoint)
src/hooks	useApolice e useOpcoes (busca de dados)
src/utils/format.js	formatação de moeda, datas e máscara de CPF

Toda chamada à API passa por `request`, que define os cabeçalhos, converte o corpo para JSON e, em caso de erro, lança um `ApiError` com a mensagem de cada campo, exibida abaixo do campo correspondente no formulário.

A **cotação em tempo real** funciona assim: quando todos os campos estão preenchidos, o formulário aguarda 400ms sem alterações (*debounce*) e chama `/api/apolices/cotacao`. Se o usuário continuar digitando, a requisição anterior é cancelada com `AbortController`, evitando que uma resposta antiga substitua a mais recente. A busca da lista usa a mesma técnica.

9 Testes

São 37 testes automatizados (PHPUnit), executados com `php artisan test` na pasta `backend`.

- **Unitários:** `CalculadoraPremioViagemTest` verifica cinco combinações de plano, destino e idade; `CpfTest` verifica CPFs válidos e inválidos; `MetricsTest` verifica as fórmulas da dashboard.
- **De API:** `ApoliceApiTest` faz requisições reais e verifica cadastro, reaproveitamento do segurado, busca e paginação, edição com recálculo, regra da apólice cancelada, exclusão lógica, mensagens de validação, limites da vigência, cotação e resumo.
- **Da dashboard:** `DashboardApiTest` verifica os indicadores de cada visão, o funil, o ROI de campanha, a sinistralidade e o seeder de demonstração.

Nos testes de API a data atual é fixada com `Carbon::setTestNow('2026-09-20')`, para que o resultado não dependa do dia em que o teste é executado, e o banco é recriado a cada teste (`RefreshDatabase`).

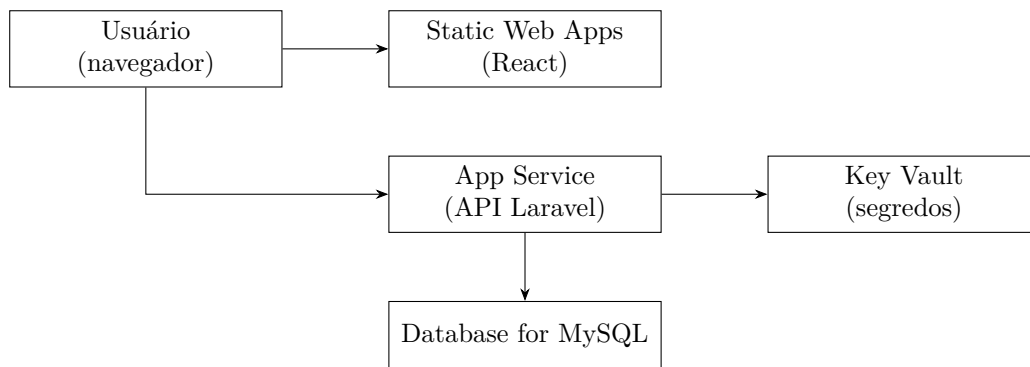
10 Execução e implantação

10.1 Com Docker

```
docker compose up -d --build
docker compose exec api php artisan db:seed      (dados de exemplo)
```

Aplicação em `http://localhost:3000` e API em `http://localhost:8000/api`.

10.2 Na Azure



- **Static Web Apps** hospeda o React compilado.
- **App Service** executa a API.
- **Azure Database for MySQL** armazena os dados.
- **Key Vault** guarda a senha do banco e a `APP_KEY`, lidas pelo App Service via *Managed Identity*.

11 Trabalhos futuros

- Autenticação e perfis de acesso (Laravel Sanctum).
- Endosso: histórico das alterações de cada apólice.
- Infraestrutura como código (Bicep).
- Testes automatizados no frontend.